Design and Analysis of Algorithms
CO2 – Debugging and Optimisation
Task: SentinelOps SOC Alert-Triage Module

(a) Reproduce and Identify the Defects

Expected vs actual output on the test data

Function	Expected output	Actual output (buggy module)
sort_alerts	[(A2,9), (A4,9), (A3,5), (A1,3), (A5,1)]	[(A4,9), (A3,5), (A3,5), (A5,1), (A5,1)] – A1 and A2 lost, A3 and A5 duplicated, equal-severity order wrong
is_blocked (10.0.0.9, 10.0.0.5, 8.8.8.8)	True, True, False	False, False, False – blocked IPs get through
find_signature("FAILFAIXFAIL", "FAIL")	[0, 8]	[0, 4] – last occurrence (8) missed, false hit at 4
closest_logins	((0, 5), 1.4142…) i.e. (2,3) and (3,4)	((0, 2), 60.44…) – this is the FARTHEST pair

The nine defects

#	Function	Faulty line	Nature of the error
1	sort_alerts	while j > 0 and …	Off-by-one loop guard: index 0 can never be shifted, so the element at index 0 is overwritten (data loss).
2	sort_alerts	alerts[j] = key	Key is placed at j instead of j + 1 (the free slot), overwriting an element and creating duplicates.
3	is_blocked	return False (inside if ==)	Logic error: a match must return True. As written the function can never return True.
4	is_blocked	else: return False (on the for)	Misleading/redundant for-else; it hides defect 3 because every path returns False. Should be a plain return False after the loop.
5	find_signature	for i in range(n - m)	Off-by-one: there are n − m + 1 windows; the last window (end of log) is never examined.
6	find_signature	if j == m - 1: hits.append(i)	Wrong match test: after a break at the last character j is also m − 1, so a mismatch on the last character is reported as a hit (false signature).

#	Function	Faulty line	Nature of the error
7	closest_logins	min_dist = 0	Wrong initial value: a minimum must start at infinity, otherwise nothing can ever be smaller.
8	closest_logins	for j in range(len(events))	Pairs with j = i (distance 0) and repeated pairs (i,j)/(j,i) are examined: double work and self-pairs.
9	closest_logins	if d > min_dist	Wrong comparison direction: finds the maximum (farthest) pair instead of the minimum.

Note: defects 3 and 4 are both in is_blocked (the wrong return value and the redundant for-else that makes the function always return False). With the else attached to the if instead of the for, the function would additionally return after checking only the first item.

(b) Root Causes and Systematic Debugging

1. sort_alerts (defects 1 and 2)

Root cause: an insertion sort shifts larger-priority elements one place right and then drops the key into the hole at j + 1. With the guard j > 0 the loop stops while j = 0 still has to be compared, and writing to alerts[j] instead of alerts[j+1] overwrites a live element. Both mistakes destroy data, which explains the disappearing and duplicated alerts. Equal severities are handled correctly by the strict < (equal items are never shifted, so arrival order is preserved).

Trace of the BUGGY code on [A1:3, A2:9, A3:5, A4:9, A5:1]:

i	key	List after the pass
1	A2 (9)	j = 0 so loop skipped; alerts[0] = A2 → [A2, A2, A3, A4, A5] (A1 lost, A2 duplicated)
2	A3 (5)	9 < 5 false; alerts[1] = A3 → [A2, A3, A3, A4, A5] (second A2 overwritten)
3	A4 (9)	shifts A3, A3 right; stops at j = 0; alerts[0] = A4 → [A4, A3, A3, A3, A5]
4	A5 (1)	5 < 1 false; alerts[3] = A5 → [A4, A3, A3, A5, A5] ← observed wrong output

Trace of the CORRECTED code (j >= 0, insert at j + 1):

i	key	Comparisons	List after the pass
start	–	–	[A1:3, A2:9, A3:5, A4:9, A5:1]
1	A2 (9)	1	A1 (3) < 9 → shift A1; j = −1; place at 0 → [A2, A1, A3, A4, A5]
2	A3 (5)	2	A1 < 5 shift; A2 (9) < 5 no; place at 1 → [A2, A3, A1, A4, A5]

i	key	Comparisons	List after the pass
3	A4 (9)	3	A1 < 9 shift; A3 < 9 shift; A2 (9) < 9 false (strict, so A2 stays ahead: stable); place at 1 → [A2, A4, A3, A1, A5]
4	A5 (1)	1	A1 (3) < 1 false; place at 4 → [A2, A4, A3, A1, A5]

Result: [(A2,9), (A4,9), (A3,5), (A1,3), (A5,1)] = expected. Total 7 comparisons.

2. is_blocked (defects 3 and 4)

Root cause: the success branch returns False, and every other path also returns False, so the function is a constant False. The blocked addresses 10.0.0.9 and 10.0.0.5 therefore pass. Fix: return True on a match and return False only after the whole list has been scanned. Re-test: True, True, False for the three queries.

3. find_signature (defects 5 and 6)

Root cause 5: with n = 12 and m = 4 there are 9 windows (i = 0…8) but range(8) visits only i = 0…7, so the occurrence at index 8 is never seen. Root cause 6: when the inner loop breaks at j = 3 (mismatch on the last character) the test j == m − 1 is still true, so window 4 ("FAIX") is wrongly reported. Fix: use for/else so a hit is recorded only if the inner loop finishes without a break, and iterate range(n − m + 1).

Alignment table for log = FAILFAIXFAIL, pattern = FAIL (corrected code):

i	Window	Comparisons made	Count	Result
0	FAIL	F=F, A=A, I=I, L=L	4	match → hit 0
1	AILF	A≠F	1	mismatch
2	ILFA	I≠F	1	mismatch
3	LFAI	L≠F	1	mismatch
4	FAIX	F=F, A=A, I=I, X≠L (break at j=3)	4	mismatch (buggy code reported a hit here)
5	AIXF	A≠F	1	mismatch
6	IXFA	I≠F	1	mismatch
7	XFAI	X≠F	1	mismatch
8	FAIL	F=F, A=A, I=I, L=L	4	match → hit 8 (buggy code never reached i = 8)

Result [0, 8]; 18 character comparisons in total.

4. closest_logins (defects 7, 8 and 9)

Root cause: the minimum starts at 0 and the test is d > min_dist, so the code keeps the largest distance ever seen (the farthest pair, (0,2) with d = √3653 ≈ 60.44 km). The inner loop starts at 0, so each pair is computed twice and every event is compared with itself (d = 0). Fix: best = ∞, j in range(i + 1, n), test d < best.

Squared distances of all 15 pairs (test data):

Pair	d^2	Pair	d^2	Pair	d^2
0–1	829	1–2	1184	2–4	2384
0–2	3653 (max)	1–3	890	2–5	3485
0–3	13	1–4	400	3–4	130
0–4	149	1–5	757	3–5	13
0–5	2 (min)	2–3	3626	4–5	117

Minimum $d^2 = 2 \rightarrow$ pair (0, 5), distance $\sqrt{2} \approx 1.4142$ km. The maximum $d^2 = 3653$ is what the buggy code returned.

Boundary tests (buggy vs corrected)

Test	Expected	Buggy module	Corrected module
sort: equal severities [a2, b2, c2]	[a, b, c] (same order)	[b, c, c] ✗	[a, b, c] ✓
sort: already sorted [c3, b2, a1]	[c, b, a]	[b, a, a] ✗	[c, b, a] ✓
sort: empty / single	[] / [x]	[] / [x] ✓	✓
is_blocked: empty blocklist	False	False (by luck)	False ✓
is_blocked: IP is last element	True	False ✗	True ✓
signature at very end: xxxxFAIL	[4]	[] ✗	[4] ✓
pattern equals log: FAIL / FAIL	[0]	[] ✗	[0] ✓
pattern longer than log: AB / ABC	[]	[] (by luck)	[] ✓ (explicit guard)
overlapping: AAAA / AA	[0, 1, 2]	[0, 1] ✗	[0, 1, 2] ✓
closest: 0 or 1 events	(None, ∞)	(None, 0) ✗	(None, ∞) ✓
closest: exactly two events	((0,1), d)	✓	✓
closest: duplicate location	((0,1), 0.0)	self pairs hide it ✗	✓

(c) Optimisation of the Corrected Module

Sorting alerts

- Insertion sort costs $\Theta(n + I)$ where I is the number of inversions. For nearly sorted input I is small, so it is close to linear and genuinely suitable. For random input it needs about $n^2/4$ comparisons, which for 50,000 alerts is $\approx 6.25 \times 10^8$ – too slow.
- Production alerts are only sometimes nearly sorted, so use Python's Timsort (list.sort(key=…, reverse=True)): stable, O(n log n) worst case, and ≈ O(n) on nearly sorted data because it detects existing runs. reverse=True keeps equal-severity alerts in arrival order. Output is identical to the corrected insertion sort.

Repeated blocklist lookups

- A sequential search costs O(b) per query. Build a hash set once (O(b)) and each lookup is O(1) on average, so Q queries cost O(b + Q) instead of O(Qb). Alternative if ordering is needed: sorted list + binary search, O(log b) per query. (For CIDR ranges, use a prefix trie or the ipaddress module.)

Signature scan

- Keep the brute-force scan but remove redundant work: iterate exactly $n - m + 1$ windows, stop a window at its first mismatch, use for/else instead of the extra $j == m - 1$ test on every window, and return immediately for $m = 0$ or $m > n$. Worst case stays $\Theta(nm)$; best case is $n - m + 1$ comparisons.

Closest pair of logins

- Only pairs with $i < j$: halves the pair count, from n^2 to $n(n - 1)/2$.
- Compare squared distances; since sqrt is monotonic the closest pair is the same. Take one square root at the very end (from n^2 to 1 square root).
- Pruning: visit events sorted by x. For event i scan forward and stop as soon as $(xj - xi)^2 >$ best, because every later event is even farther in x. The check is strict (>) so ties are not lost, and ties are broken by the smallest (i, j), which makes the output identical to the corrected brute force.

(d) Analysis Before and After

Operation counts and complexity

Function	Before (buggy)	After (corrected + optimised)	Notes
sort_alerts	Comparisons: best $n-1$, avg $\approx n^2/4$, worst $n(n-1)/2 \rightarrow O(n^2)$; wrongly loses data	Timsort: $\approx n \log_2 n$ comparisons worst case $\rightarrow O(n \log n)$; $O(n)$ if nearly sorted	$n = 50{,}000$: 6.25×10^8 vs $\approx 7.8\times10^5$
is_blocked	$\leq b$ comparisons per query $\rightarrow O(Qb)$ for Q queries	$O(b)$ build, $O(1)$ average per query $\rightarrow O(b + Q)$	$b = 10{,}000$: $\approx 7{,}650$ comparisons per query vs 1 hash lookup

Function	Before (buggy)	After (corrected + optimised)	Notes
find_signature	$(n-m)\cdot m$ worst case, extra j == m−1 test per window → $\Theta(nm)$	$(n-m+1)\cdot m$ worst case, n−m+1 best case → $\Theta(nm)$; no redundant tests	Same asymptotics; correct and leaner
closest_logins	n^2 distance computations, n^2 square roots → $\Theta(n^2)$	Sort O(n log n) + pruned scan; 1 square root. Worst case (all same x) still $\Theta(n^2)$ but with half the pairs	n = 8000: 64,000,000 vs 31,996,000 (plain) vs ≈ 11,380 (pruned, random)

Measurement 1 – closest_logins distance computations

Random points uniformly in a 1000 × 1000 km square (seed 42); worst-case column uses events that all share one x coordinate, so nothing can be pruned.

Events n	Original n^2 (with sqrt)	Corrected brute $n(n-1)/2$	Optimised (random)	Optimised (same-x worst)
1,000	1,000,000	499,500	1,778	499,500
2,000	4,000,000	1,999,000	3,877	1,999,000
4,000	16,000,000	7,998,000	3,904	7,998,000
8,000	64,000,000	31,996,000	11,380	31,996,000

Agreement with analysis: the brute-force counts match n^2 and $n(n-1)/2$ exactly (doubling n multiplies the count by 4). The pruned version uses only a few thousand computations on random data (more than 99.9% fewer at n = 8000), but on the adversarial same-x input it degrades to exactly $n(n-1)/2$, confirming that pruning improves the typical case, not the worst-case bound. The count for n = 4000 is slightly lower than for 2000 only because each point set is a different random sample; the scan depends on how close the nearest pair happens to be.

Measurement 2 – insertion sort comparisons

n	Random	Nearly sorted (2% adjacent swaps)	$n^2/4$ (predicted avg)	n−1 (predicted best)	$n(n-1)/2$ (worst)
1,000	253,093	1,019	250,000	999	499,500
2,000	988,503	2,037	1,000,000	1,999	1,999,000
5,000	6,143,498	5,095	6,250,000	4,999	12,497,500

Agreement with analysis: random data lies within about 1–2% of $n^2/4$, and nearly sorted data is only slightly above n − 1 (extra ≈ number of inversions, here about n/50). So insertion sort is excellent for nearly sorted alerts and unacceptable for random ones. Timsort on random data

used 714,769 comparisons for n = 50,000 ($n \log_2 n \approx 780{,}000$), against $\approx 6.25 \times 10^8$ expected for insertion sort.

Measurement 3 – blocklist lookups

b = 10,000 IPs: a list scan averaged 7,654 comparisons per query (≈ b/2 for hits, b for misses); 2,000 list queries took 0.29 s while 50,000 set lookups took 0.0023 s – roughly three orders of magnitude faster per query, matching O(b) versus O(1).

(e) Production-Quality Code

Final module (sentinelops.py). The corrected reference versions are kept so the test harness can prove the optimised functions give identical output.

```
"""SentinelOps SOC alert-triage module (CSA06 - CO2 Debugging and Optimisation Task).

Functions
---------
sort_alerts           -- severity sort, highest first, stable (optimised: Timsort)
insertion_sort_alerts  -- corrected insertion sort (reference / baseline)
Blocklist, is_blocked  -- hash-set blocklist lookup (optimised) and one-off wrapper
is_blocked_linear      -- corrected sequential search (reference)
find_signature         -- brute-force substring search, all occurrences
closest_logins         -- brute-force closest pair with pruning (optimised)
closest_logins_brute   -- corrected plain brute force (reference)
"""
import math


# ---------------------------------------------------------------- sorting
def insertion_sort_alerts(alerts):
    """Corrected insertion sort: severity descending, stable, in place.

    Fixes: loop guard j >= 0 (so index 0 can be shifted) and the key is
    inserted at j + 1 (the slot left free), not at j.
    Strict '<' means equal severities are never swapped -> arrival order kept.
    """
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j >= 0 and alerts[j][1] < key[1]:   # fix 1: j >= 0
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j + 1] = key                       # fix 2: j + 1
    return alerts


def sort_alerts(alerts):
```

```
    """Sort alerts by severity, highest first, equal severities in arrival order.

    list.sort is Timsort: stable, O(n log n) worst case and O(n) on
    (nearly) sorted input. reverse=True keeps the stability guarantee.
    """
    alerts.sort(key=lambda alert: alert[1], reverse=True)
    return alerts


# ------------------------------------------------------------ blocklist
def is_blocked_linear(blocklist, ip):
    """Corrected sequential search: True if ip is in blocklist."""
    for item in blocklist:
        if item == ip:
            return True           # fix 3: a match means BLOCKED
    return False                  # fix 4: plain return after the loop


class Blocklist:
    """Blocklist with O(1) average lookups (built once, queried many times)."""

    def __init__(self, ips):
        self._ips = set(ips)      # O(b) one-time build

    def __contains__(self, ip):
        return ip in self._ips

    def is_blocked(self, ip):
        """Return True if ip is on the blocklist."""
        return ip in self._ips


def is_blocked(blocklist, ip):
    """One-off lookup; for repeated queries build a Blocklist once instead."""
    return ip in blocklist if isinstance(blocklist, (set, frozenset, Blocklist)) \
        else is_blocked_linear(blocklist, ip)


# ------------------------------------------------------------ signature
def find_signature(log, pattern):
    """Return every start index where pattern occurs in log (overlaps allowed).

    Brute force. Fixes: window range is n - m + 1 (last window was skipped)
    and a hit is recorded only when the inner loop finishes WITHOUT a
    mismatch (for/else), instead of testing j == m - 1 after a break.
    An empty pattern returns [] by definition (no meaningful signature).
```

```
    """
    n, m = len(log), len(pattern)
    hits = []
    if m == 0 or m > n:
        return hits
    for i in range(n - m + 1):                  # fix 5: +1
        for j in range(m):
            if log[i + j] != pattern[j]:
                break                           # first mismatch ends window
        else:                                   # fix 6: no break -> full match
            hits.append(i)
    return hits


# ---------------------------------------------------------- closest pair
def closest_logins_brute(events):
    """Corrected plain brute force: returns ((i, j), distance), i < j.

    Fixes: j starts at i + 1 (no self / repeated pairs), minimum starts at
    infinity, and the test is '<' (we want the CLOSEST pair).
    Fewer than two events -> (None, inf).
    """
    best = math.inf                             # fix 7: start at infinity
    pair = None
    for i in range(len(events)):
        for j in range(i + 1, len(events)):     # fix 8: j > i
            d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
                          (events[i][1] - events[j][1]) ** 2)
            if d < best:                        # fix 9: '<'
                best = d
                pair = (i, j)
    return pair, best


def closest_logins(events):
    """Closest pair of login locations, same result as closest_logins_brute.

    Optimisations: (1) i < j only; (2) compare squared distances and take one
    square root at the end; (3) events are visited in x order and the inner
    scan stops as soon as dx^2 > best, because no later point can be closer.
    Ties are broken by the smallest (i, j) so output equals the brute force.
    """
    n = len(events)
    if n < 2:
        return None, math.inf
    order = sorted(range(n), key=lambda k: events[k][0])
```

```
    best = math.inf
    pair = None
    for a in range(n):
        i = order[a]
        xi, yi = events[i]
        for b in range(a + 1, n):
            j = order[b]
            xj, yj = events[j]
            dx2 = (xj - xi) ** 2
            if dx2 > best:                  # prune: rest are farther in x
                break
            d2 = dx2 + (yj - yi) ** 2
            cand = (i, j) if i < j else (j, i)
            if d2 < best or (d2 == best and cand < pair):
                best, pair = d2, cand
    return pair, math.sqrt(best)
```

Test harness (test_sentinelops.py) – output: PASS test_sort, PASS test_blocklist, PASS test_signature, PASS test_closest, All tests passed.

```
"""Test harness for sentinelops.py - run: python test_sentinelops.py"""
import math, random
from sentinelops import *

def test_sort():
    alerts = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
    want = [("A2", 9), ("A4", 9), ("A3", 5), ("A1", 3), ("A5", 1)]
    assert insertion_sort_alerts(list(alerts)) == want
    assert sort_alerts(list(alerts)) == want
    for f in (insertion_sort_alerts, sort_alerts):
        assert f([]) == []                          # empty
        assert f([("X", 4)]) == [("X", 4)]          # single
        eq = [("a", 2), ("b", 2), ("c", 2)]
        assert f(list(eq)) == eq                    # equal severities: arrival order
        assert f([("a", 1), ("b", 2), ("c", 3)]) == [("c", 3), ("b", 2), ("a", 1)]  # ascending input
        assert f([("c", 3), ("b", 2), ("a", 1)]) == [("c", 3), ("b", 2), ("a", 1)]  # already sorted
    for _ in range(200):                            # optimised == corrected reference
        data = [(f"A{k}", random.randint(1, 5)) for k in range(random.randint(0, 40))]
        assert sort_alerts(list(data)) == insertion_sort_alerts(list(data))

def test_blocklist():
    bl = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
    for q, want in [("10.0.0.9", True), ("10.0.0.5", True), ("172.16.4.2", True), ("8.8.8.8", False)]:
        assert is_blocked_linear(bl, q) is want
        assert is_blocked(bl, q) is want
        assert Blocklist(bl).is_blocked(q) is want
    assert is_blocked_linear([], "1.1.1.1") is False        # empty blocklist
```

```
    assert Blocklist([]).is_blocked("1.1.1.1") is False
    assert is_blocked_linear(["1.1.1.1"], "1.1.1.1") is True # single element, last element

def test_signature():
    assert find_signature("FAILFAIXFAIL", "FAIL") == [0, 8]  # last window + no false hit at 4
    assert find_signature("FAIL", "FAIL") == [0]            # pattern == log
    assert find_signature("AB", "ABC") == []                # pattern longer than log
    assert find_signature("", "A") == []                    # empty log
    assert find_signature("abc", "") == []                  # empty pattern
    assert find_signature("AAAA", "AA") == [0, 1, 2]        # overlapping hits
    assert find_signature("xxxxFAIL", "FAIL") == [4]        # pattern at very end
    assert find_signature("FAIL", "F") == [0]
    for _ in range(300):                                    # compare with str.find
        log = "".join(random.choice("ab") for _ in range(random.randint(0, 30)))
        pat = "".join(random.choice("ab") for _ in range(random.randint(1, 4)))
        want = [i for i in range(len(log) - len(pat) + 1) if log.startswith(pat, i)]
        assert find_signature(log, pat) == want

def test_closest():
    ev = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]
    (p, d) = closest_logins(ev)
    assert p == (0, 5) and abs(d - math.sqrt(2)) < 1e-12
    assert closest_logins_brute(ev) == closest_logins(ev)
    assert closest_logins([]) == (None, math.inf)           # fewer than two events
    assert closest_logins([(1, 1)]) == (None, math.inf)
    assert closest_logins([(1, 1), (4, 5)]) == ((0, 1), 5.0) # exactly two
    assert closest_logins([(1, 1), (1, 1), (9, 9)])[0] == (0, 1)  # duplicate location, d = 0
    ev = [(0, 0), (1, 0), (5, 5), (6, 5)]                   # tie: first (i, j) wins
    assert closest_logins(ev) == closest_logins_brute(ev) == ((0, 1), 1.0)
    for _ in range(300):                                    # optimised == brute force
        ev = [(random.randint(0, 20), random.randint(0, 20)) for _ in range(random.randint(0, 40))]
        assert closest_logins(ev) == closest_logins_brute(ev)

if __name__ == "__main__":
    for t in (test_sort, test_blocklist, test_signature, test_closest):
        t(); print("PASS", t.__name__)
    print("All tests passed")
```